SOFTWARE ENGINEERING

NOTEBOOK

Table of Contents

CHAPTER 1

Algorithms

1 Time Complexity

Time complexity for an algorithm has tree different facets

- The *worst-case* complexity of the algorithm is the function defined by the maximum number of steps taken in any instance of size n.
- The *best-case* complexity of the algorithm is the function defined by the minimum number of steps taken in any instance of size n.
- The *average-case* complexity or expected time of the algorithm, which is the function defined by the average number of steps over all instances of size n.

1.1 Big O notation

$$f(n) = O(g(n)) \quad \text{if} \quad 0 \leq f(n) \leq cg(n) \forall n \geq n_0 \tag{1.1}$$

For all values $n >= n_0$, $f(n)$ is on or below $cg(n)$.
A constant multiple of $g(n)$ is an *asimptotic upper bound* for $f(n)$.

1.2 Omega notation

$$f(n) = \Omega(g(n)) \quad \text{if} \quad 0 \leq cg(n) \leq f(n) \forall n \geq n_0 \tag{1.2}$$

For all values $n >= n_0$, $f(n)$ is on or abow $cg(n)$.
A constant multiple of $g(n)$ is an *asimptotic lower bound* for $f(n)$.

1.3 Theta notation

$$f(n) = \Theta(g(n)) \quad \text{if} \quad 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \forall n \geq n_0 \tag{1.3}$$

For all values $n >= n_0$, $f(n)$ is equal to $g(n)$ within a constant factor.
$g(n)$ is an *asimptotically tight bound* for $f(n)$.

1.4 Meaning of previous notations - Insertion sort case

Big O notation
describes an *upper bound*. When we use it to *bound the worst-case running time* of an algorithm, we have a bound on the running time of the algorithm on *every input*.
Thus, the $O(n^2)$ bound on worst-case running time of *insertion sort* also applies to its running time on every input.
Omega notation
describes a *lower bound*. When we say that the running time (no modifier) of an algorithm is $\Omega(g(n))$, we mean that *no matter what particular input of size n is chosen* for each value of n, the running time on that input is at least a constant times $g(n)$, for sufficiently large n.
Equivalently, we are giving a *lower bound on the best-case running time* of an algorithm.
For example, the best-case running time of insertion sort is $\Omega(n)$, which implies that the running time of insertion sort is $\Omega(n)$.
The *running time of insertion sort therefore belongs to both $\Omega(n)$ and $O(n^2)$*, since it falls anywhere between a linear function of n and a quadratic function of n. Moreover, these bounds are asymptotically as tight as possible: for instance, the running time of insertion sort is not $\Omega(n^2)$, since there exists an input for which insertion sort runs in $\Theta(n)$ time (e.g., when the input is already sorted). It is not contradictory, however, to say that the worst-case running time of insertion sort is $\Omega(n^2)$, since there exists an input that causes the algorithm to take $\Omega(n^2)$ time.
Theta notation
describes a *tight bound*. The $\Theta(n^2)$ notation bound on the worst-case running time of insertion sort, however, does not imply a $\Theta(n^2)$ bound on the running time of insertion

sort on every input. For example, when the input is already sorted, insertion sort runs in $\Theta(n)$ time. Technically, it is an abuse to say that the running time of insertion sort is $O(n^2)$, since for a given n, the actual running time varies, depending on the particular input of size n. When we say "the running time is $O(n^2)$", we mean that there is a function $f(n)$ that is $O(n^2)$ such that for any value of n, no matter what particular input of size n is chosen, the running time on that input is bounded from above by the value $f(n)$. Equivalently, we mean that the worst-case running time is $O(n^2)$.

1.5 Running times of well known algorithms

Algorithm	Best	Average	Worst	Worst Space Complexity
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Mergesort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Timsort	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Heapsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Shell Sort	$O(n)$	$O((n \log n)^2)$	$O((n \log n)^2)$	$O(1)$
Bucket Sort	$O(n+k)$	$O(n+k)$	$O(n^2)$	$O(n)$
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$

Verbal description of the previous sorting algorithms:

- Quicksort
  Efficient, general-purpose sorting algorithm, slightly faster than merge sort and heapsort for randomized data, particularly on larger distributions.
  It selects a pivot element from the array and partitions the other elements into two sub-arrays, according to whether they are less than or greater than the pivot.
  For this reason, it is sometimes called partition-exchange sort.
  The sub-arrays are then sorted recursively.

- Mergesort
  First, divide the list into the smallest unit (1 element), then compare each element with the adjacent list to sort and merge the two adjacent lists.
  Finally, all the elements are sorted and merged.

- Timsort
  a hybrid, stable sorting algorithm, derived from merge sort and insertion sort, designed to perform well on many kinds of real-world data. It was implemented by Tim Peters in 2002 for use in the Python programming language. The algorithm finds subsequences of the data that are already ordered (runs) and uses them to sort the remainder more efficiently. This is done by merging runs until certain criteria are fulfilled. Timsort was Python's standard sorting algorithm from version 2.3 to version 3.10

- Heapsort
  comparison-based sorting algorithm which can be thought of as "an implementation of selection sort using the right data structure."[3] Like selection sort, heapsort divides its input into a sorted and an unsorted region, and it iteratively shrinks the unsorted region by extracting the largest element from it and inserting it into the sorted region. Unlike selection sort, heapsort does not waste time with a

linear-time scan of the unsorted region; rather, heap sort maintains the unsorted region in a heap data structure to efficiently find the largest element in each step.

- Bubble Sort
  a simple sorting algorithm that repeatedly steps through the input list element by element, comparing the current element with the one after it, swapping their values if needed. These passes through the list are repeated until no swaps have to be performed during a pass, meaning that the list has become fully sorted

- Insertion Sort
  iterates, consuming one input element each repetition, and grows a sorted output list. At each iteration, insertion sort removes one element from the input data, finds the location it belongs within the sorted list, and inserts it there. It repeats until no input elements remain.

- Selection Sort
  repeatedly identifies the smallest remaining unsorted element and puts it at the end of the sorted portion of the array

- Shell Sort

- Bucket Sort
  Each bucket is then sorted individually, either using a different sorting algorithm, or by recursively applying the bucket sorting algorithm. It is a distribution sort, a generalization of pigeonhole sort that allows multiple keys per bucket, and is a cousin of radix sort in the most-to-least significant digit flavor. Bucket sort can be implemented with comparisons and therefore can also be considered a comparison sort algorithm. The computational complexity depends on the algorithm used to sort each bucket, the number of buckets to use, and whether the input is uniformly distributed.

- Radix Sort

1.6 Methods for solving the recurrence

There are three methods for solving recurrences:

- the substitution method
- the recursion-tree method
- the master theorem

2 Space Complexity

Algorithms Bibliography

[90] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley.

[91] Steven S. Skiena. *The Algorithm Design Manual*. Springer.

[92] *Quicksort*. Wikipedia.

[93] *Merge Sort*. Wikipedia.

[94] *Timsort*. Wikipedia.

[95] *Heapsort*. Wikipedia.

[96] *Bubble Sort*. Wikipedia.

[97] *Insertion Sort*. Wikipedia.

[98] *Selection Sort*. Wikipedia.

[99] *Shell Sort*. Wikipedia.

[100] *Bucket Sort*. Wikipedia.

[101] *Radix Sort*. Wikipedia.